1. Fetching & Displaying JSON Data

◇ What is JSON?

JSON (JavaScript Object Notation) is a **data format** used to send data from backend → frontend.

Example:

```
[
  { "id": 1, "name": "Ali", "email": "ali@gmail.com" },
  { "id": 2, "name": "Sara", "email": "sara@gmail.com" }
]
```

👉 This is an **array of objects**

◇ What happens in React?

Step-by-step flow:

1. Fetch data from API
2. Store it in state
3. Display it on UI

---

2. Using `map()` to Render Lists Dynamically

◇ What is `map()`?

`map()` is a JavaScript function used to:

- Loop through an array
- Convert each item into JSX (UI)

◇ Syntax:

```
array.map((item) => {
  return something
})
```

☑ Example:

```
const users = [
  { id: 1, name: "Ali" },
  { id: 2, name: "Sara" }
]

const App = () => {
  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  )
}
```

🔥 Important Concept:

- React does NOT allow direct loops like `for`
- So we use `map()` to render lists

🔥 Why `key` is important?

```
<p key={user.id}>{user.name}</p>
```

👉 React uses `key` to:

- Identify elements
- Improve performance
- Update UI efficiently

📓 3. Axios (Deep Explanation)

◇ What is Axios?

Axios is a **JavaScript library** used to make HTTP requests.

◇ Why use Axios?

- Simple syntax
- Handles JSON automatically
- Better than fetch for beginners

◇ Install:

```
npm install axios
```

🗄️ 4. async/await (Important Concept)

◇ Problem (Old way):

```
axios.get(url)
  .then(res => console.log(res.data))
```

👉 Hard to read when multiple APIs are used

◇ Solution (async/await):

```
const fetchData = async () => {
  const res = await axios.get(url)
  console.log(res.data)
}
```

🔥 Key Concepts:

Keyword	Explanation
async	Makes function asynchronous
await	Waits for response
try/catch	Handles errors

☑️ Example with error handling:

```
const fetchData = async () => {
  try {
    const res = await axios.get("API_URL")
    console.log(res.data)
  } catch (error) {
    console.log(error)
  }
}
```

🟫 5. React + Axios + async/await (Complete Flow)

☑️ Full Working Example:

```
import { useEffect, useState } from "react"
import axios from "axios"

const App = () => {

  const [users, setUsers] = useState([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    fetchUsers()
  }, [])

  const fetchUsers = async () => {
    try {
      const res = await
axios.get("https://jsonplaceholder.typicode.com/users")
      setUsers(res.data)
    } catch (error) {
      console.log(error)
    } finally {
      setLoading(false)
    }
  }

  if (loading) return <h2>Loading...</h2>

  return (
    <div>
      <h2>User List</h2>

      {users.map(user => (
        <div key={user.id}>
          <h3>{user.name}</h3>
          <p>{user.email}</p>
        </div>
      ))}

    </div>
  )
}

export default App
What is this?
```

🔥 Flow Explanation:

1. Component loads

2. `useEffect()` runs
3. API function is called
4. Axios fetches data
5. Data stored in state
6. `map()` displays it

🟫 6. Axios Methods (ALL with Explanation)

🟢 6.1 GET (Read Data)

👉 Used to fetch data

```
const getUsers = async () => {
  const res = await axios.get("/users")
  console.log(res.data)
}
```

🟡 6.2 POST (Create Data)

👉 Used to send new data

```
const addUser = async () => {
  const user = {
    name: "Ali",
    email: "ali@gmail.com"
  }

  const res = await axios.post("/users", user)
}
```

🔵 6.3 PUT (Full Update)

👉 Replaces entire object

```
const updateUser = async () => {
  const updatedUser = {
    name: "New Name",
    email: "new@gmail.com"
  }
```

```
  await axios.put("/users/1", updatedUser)
}
```

🔄 6.4 PATCH (Partial Update)

👉 Updates only specific fields

```
const updateName = async () => {
  await axios.patch("/users/1", {
    name: "Updated Name"
  })
}
```

🟢 6.5 DELETE (Remove Data)

👉 Deletes data

```
const deleteUser = async () => {
  await axios.delete("/users/1")
}
```

📦 7. Handling Responses

```
const res = await axios.get(url)

res.data     // actual data
res.status   // HTTP status
```

📦 8. Error Handling (Important)

```
try {
  const res = await axios.get(url)
} catch (error) {
  console.log(error.message)
}
```

📦 9. Loading State (Best Practice)

```
const [loading, setLoading] = useState(true)
if (loading) return <h2>Loading...</h2>
```

👉 Improves user experience

10. Axios vs Postman

Axios	Postman
Used in code	Used for testing
Shows UI data	No UI
Part of app	External tool